Ubuntu Shell Scripting A to Z

Complete Guide: Basics to Advanced, with Backup, Automation, Monitoring and Real-World Scripts

```
$ #!/bin/bash
$ echo "Hello, Ubuntu!"
$ tar -czf backup.tar.gz /home
$ crontab -e
```

Tested style for Ubuntu 22.04 / 24.04 LTS and Bash 5.x

Table of Contents

Chapter 1: Introduction to Shell Scripting 5

1.1 Why learn shell scripting? . . . 5
1.2 What is a shell? . . . 5
1.3 Your first script . . . 5
1.4 The shebang line . . . 6
1.5 Different ways to run a script . . . 6
1.6 Comments . . . 6
1.7 Exit status . . . 6
1.8 A good script template . . . 7
1.9 Naming and location conventions . . . 7
1.10 Useful editors on Ubuntu . . . 7

Chapter 2: Variables and Data 9

2.1 Creating and using variables . . . 9
2.2 Quotes: double, single, backslash . . . 9
2.3 Command substitution . . . 9
2.4 Environment variables . . . 9
2.5 Special (built-in) variables . . . 10
2.6 Readonly, unset and declare . . . 11
2.7 Parameter expansion (very powerful) . . . 11
2.8 Reading values from the user . . . 12
2.9 Variable scope . . . 12
2.10 Naming convention summary . . . 12

Chapter 3: Input, Output and Redirection 13

3.1 echo . . . 13
3.2 printf (formatted output) . . . 13
3.3 read (taking input) . . . 14
3.4 Redirection . . . 14
3.5 Here documents and here strings . . . 15
3.6 Pipes and tee . . . 15
3.7 File descriptors and exec . . . 15
3.8 Colors in the terminal . . . 15
3.9 Progress bar and spinner . . . 16

Chapter 4: Operators and Expressions 17

4.1 Arithmetic . . . 17
4.2 Numeric comparison . . . 17
4.3 String comparison . . . 18
4.4 File test operators . . . 18
4.5 Logical operators . . . 18

4.6 [ ] versus [[ ]] versus (( )) . . . 19
4.7 Operator precedence quick note . . . 19
4.8 Example: simple calculator . . . 19

Chapter 5: Decision Making (if, case) 20

5.1 if / elif / else . . . 20
5.2 if with commands (no brackets) . . . 20
5.3 Common one-line checks . . . 21
5.4 case statement . . . 21
5.5 Service control menu with case . . . 22
5.6 Practical examples . . . 22
5.7 Common mistakes . . . 23

Chapter 6: Loops 24

6.1 for loop (list) . . . 24
6.2 for loop (C style) . . . 24
6.3 for loop over files and command output . . . 24
6.4 while loop . . . 25
6.5 Reading a file line by line . . . 25
6.6 until loop . . . 25
6.7 select loop (menu) . . . 26
6.8 break and continue . . . 26
6.9 Practical loop programs . . . 26
6.10 Running loop bodies in parallel . . . 27

Chapter 7: Functions 28

7.1 Defining and calling . . . 28
7.2 Arguments . . . 28
7.3 Returning values . . . 28
7.4 Local variables . . . 28
7.5 Recursion . . . 29
7.6 Useful function library . . . 29
7.7 Function tips . . . 29

Chapter 8: Arrays 31

8.1 Indexed arrays . . . 31
8.2 Looping through arrays . . . 31
8.3 Array operations . . . 31
8.4 Associative arrays (key-value) . . . 31

Chapter 9: String Manipulation 33

9.1 Basic string operations . . . 33
9.2 Checking strings . . . 33
9.3 Trimming, padding, repeating . . . 33

9.4 Splitting and joining 33
9.5 Useful string examples 33

Chapter 1: Introduction to Shell Scripting

A **shell script** is a plain text file that contains Linux commands. Instead of typing the same commands again and again in the terminal, you write them once in a file and run the file. Shell scripting is the number one skill for Ubuntu system administrators, DevOps engineers, developers and power users.

This book teaches Bash scripting step by step in simple English. Every topic has a small explanation and a working example. At the end you will find a full A to Z command reference, a cheat sheet, interview questions and practice exercises.

1.1 Why learn shell scripting?

- **Automation**: backups, updates, cleanup, reports run by themselves.
- **Time saving**: one script can do the work of 100 manual commands.
- **No extra software**: Bash is already installed on every Ubuntu system.
- **Server management**: manage hundreds of servers using the same script.
- **Career**: shell scripting is asked in almost every Linux / DevOps interview.

1.2 What is a shell?

The **shell** is a program that takes your commands, gives them to the Linux kernel and shows you the result. When you open the Terminal on Ubuntu, a shell is running. The default interactive shell on Ubuntu is **Bash** (Bourne Again SHell).

Shell	Path	Notes
bash	/bin/bash	Default login shell on Ubuntu. Most scripts use this. Has arrays, [[ ]], etc.
sh (dash)	/bin/sh	On Ubuntu /bin/sh points to dash: fast, but fewer features. Use for POSIX scripts.
zsh	/usr/bin/zsh	Modern interactive shell, many plugins (not installed by default).
ksh	/usr/bin/ksh	Korn shell. Rare on Ubuntu.
fish	/usr/bin/fish	User friendly shell, not POSIX compatible.

Check your shell

```
echo $SHELL           # your login shell
bash --version        # bash version (Ubuntu 22.04 has 5.1, 24.04 has 5.2)
cat /etc/shells       # list of valid shells
ls -l /bin/sh         # shows /bin/sh -> dash
```

1.3 Your first script

Open the terminal (Ctrl+Alt+T). Create a file with the nano editor:

```
nano hello.sh
```

Type the following, save with Ctrl+O, Enter, and exit with Ctrl+X.

hello.sh

```
#!/bin/bash
# My first script
echo "Hello, Ubuntu World!"
echo "Today is $(date)"
echo "You are logged in as: $USER"
```

Now make the file executable and run it:

```
chmod +x hello.sh
./hello.sh
```

```
Hello, Ubuntu World!
Today is Tue Oct 27 10:15:42 IST 2026
You are logged in as: ravi
```

1.4 The shebang line

The first line `#!/bin/bash` is called **shebang** (or hashbang). It tells Linux which program should run this file. Without it the current shell is used, which may behave differently.

Shebang	Meaning
`#!/bin/bash`	Run with Bash (most common, use this by default).
`#!/usr/bin/env bash`	Find bash in PATH. More portable across systems.
`#!/bin/sh`	Run with POSIX sh (dash on Ubuntu). No bash-only features!
`#!/usr/bin/python3`	Same idea for Python scripts.

1.5 Different ways to run a script

Method	Example	Notes
Execute directly	`./hello.sh`	Needs chmod +x and shebang. Runs in a new process.
Via interpreter	`bash hello.sh`	No execute permission needed.
Full path	`/home/ravi/hello.sh`	Works from any directory.
Source it	`source hello.sh OR . hello.sh`	Runs in the CURRENT shell. Variables stay after it ends.
From PATH	`hello.sh`	Copy to /usr/local/bin or ~/bin (must be in PATH).
Debug mode	`bash -x hello.sh`	Prints every command before running it.

1.6 Comments

```
# This is a single line comment
echo "Hi"          # comment after a command

: <<'COMMENT'
This is a multi-line comment.
Everything between the markers is ignored.
COMMENT
```

1.7 Exit status

Every command returns a number when it finishes. **0 means success**; any other number (1-255) means failure. The special variable `$?` holds the exit status of the last command.

```
ls /etc/passwd
echo $?        # 0 -> success
ls /nonexistent
echo $?        # 2 -> error

# Your script can return a status too
exit 0         # success
exit 1         # general error
```

Code	Meaning
0	Success
1	General error
2	Misuse of shell command / wrong arguments
126	Command found but not executable (permission problem)
127	Command not found
130	Script stopped with Ctrl+C

Code	Meaning
137	Killed with SIGKILL (kill -9)

1.8 A good script template

Use this skeleton for every new script. It includes safe options, a usage function and a main function. Each part is explained in later chapters.

template.sh

```
#!/usr/bin/env bash
#================================================================
# Name        : myscript.sh
# Description : What this script does
# Author      : Your Name
# Date        : 2026-10-27
# Usage       : ./myscript.sh [options]
#================================================================
set -euo pipefail            # stop on error, unset var, pipe error
IFS=$'\n\t'                  # safer word splitting

readonly SCRIPT_NAME="$(basename "$0")"
readonly LOG_FILE="/tmp/${SCRIPT_NAME%.sh}.log"

usage() {
    echo "Usage: $SCRIPT_NAME [-h] [-v] file"
    exit 1
}

log() {
    echo "$(date '+%F %T') [INFO] $*" | tee -a "$LOG_FILE"
}

main() {
    [[ $# -ge 1 ]] || usage
    log "Started with argument: $1"
    # ... your code here ...
    log "Finished"
}

main "$@"
```

1.9 Naming and location conventions

- Use `.sh` extension (optional but helpful): `backup.sh`, `check_disk.sh`.
- Use lowercase and underscores. Avoid spaces in file names.
- Keep personal scripts in `~/bin` or `~/scripts`. System-wide scripts go in `/usr/local/bin`.
- Do not call your script `test` (it clashes with the built-in `test` command).
- Add `~/bin` to PATH: `echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bashrc`

1.10 Useful editors on Ubuntu

Editor	Command	Level
nano	`nano file.sh`	Beginner: shortcuts shown at the bottom (^O save, ^X exit).
vim	`vim file.sh`	Advanced: i = insert, Esc then :wq = save and quit, :q! = quit without saving.
gedit / gnome-text-editor	`gedit file.sh`	GUI editor.
VS Code	`code file.sh`	Install from snap: sudo snap install code --classic

> **TIP:** Install **ShellCheck** (`sudo apt install shellcheck`). It reads your script and tells you about bugs and bad practices. Use it on every script.

Chapter 2: Variables and Data

2.1 Creating and using variables

A variable stores a value. In Bash there are **no spaces** around the `=` sign, and you read the value using `$`. Variable names can have letters, digits and underscore, but cannot start with a digit. Names are case sensitive (`name` and `NAME` are different).

variables.sh

```
name="Ravi"
age=25
city='Bhilwara'

echo "Name: $name"
echo "Age: ${age}"              # braces are safer: ${age}years
echo "Living in $city"

# WRONG examples
# name = "Ravi"      -> error: command not found (spaces!)
# 1name=abc          -> error: cannot start with digit
```

2.2 Quotes: double, single, backslash

Quote	Behaviour	Example
"double"	Variables and $( ) are expanded; spaces are kept	`echo "Hi $USER" -> Hi ravi`
'single'	Everything is literal, nothing expanded	`echo 'Hi $USER' -> Hi $USER`
\ backslash	Escapes the next character	`echo \$HOME -> $HOME`
$"..."	Locale translation (rare)	
$'..'	ANSI-C quoting: \n \t work	`echo $'a\tb\nc'`

> **WARNING:** Always put variables in double quotes: `"$file"`. Without quotes, a file name with a space (like `my notes.txt`) is split into two words and your script breaks or deletes the wrong file.

```
file="my notes.txt"
touch $file          # BAD: creates two files "my" and "notes.txt"
touch "$file"        # GOOD: creates one file
```

2.3 Command substitution

Store the output of a command in a variable using `$(command)`. The old style uses backticks, but `$( )` is better because it can be nested.

```
today=$(date +%F)
files=$(ls | wc -l)
kernel=$(uname -r)
echo "Date: $today, Files: $files, Kernel: $kernel"

# nested
echo "Size: $(du -sh "$(pwd)" | cut -f1)"
```

2.4 Environment variables

Environment variables are available to all programs started from your shell. Use `export` to make a variable an environment variable. Common ones:

Variable	Meaning	Example value
HOME	Home directory of current user	`/home/ravi`
USER	Current username	`ravi`
PWD	Present working directory	`/home/ravi/scripts`

Variable	Meaning	Example value
OLDPWD	Previous directory	`/home/ravi`
PATH	Directories searched for commands	`/usr/local/bin:/usr/bin:/bin`
SHELL	Login shell	`/bin/bash`
HOSTNAME	Computer name	`ubuntu-pc`
LANG	Language / locale	`en_IN.UTF-8`
EDITOR	Default text editor	`nano`
UID	Numeric user id (0 = root)	`1000`
RANDOM	Random number 0-32767	`21893`
SECONDS	Seconds since the shell/script started	`42`
LINENO	Current line number in script	`17`
BASH_VERSION	Bash version	`5.1.16(1)-release`
IFS	Internal Field Separator (word splitter)	`space, tab, newline`
PS1	Prompt string of terminal	`\u@\h:\w\$`

```
export MYAPP_ENV="production"     # visible to child processes
printenv | head                  # list environment variables
env | grep MYAPP                 # search one
unset MYAPP_ENV                  # remove variable

# Make permanent: add to ~/.bashrc (per user) or /etc/environment (all users)
echo 'export EDITOR=nano' >> ~/.bashrc
source ~/.bashrc
```

2.5 Special (built-in) variables

Variable	Meaning
`$0`	Name of the script
`$1 ... $9, ${10}`	Positional arguments passed to the script
`$#`	Number of arguments
`$@`	All arguments as separate words ("$@" keeps each argument intact)
`$*`	All arguments as one single string
`$?`	Exit status of the last command
`$$`	Process ID (PID) of the current script
`$!`	PID of the last background process
`$_`	Last argument of the previous command
`$-`	Current shell option flags

args.sh

```
#!/bin/bash
echo "Script name   : $0"
echo "First arg     : $1"
echo "Second arg    : $2"
echo "Total args    : $#"
echo "All args      : $@"
echo "My PID        : $$"

$ ./args.sh apple banana cherry
Script name   : ./args.sh
First arg     : apple
Second arg    : banana
Total args    : 3
All args      : apple banana cherry
My PID        : 48213
```

2.6 Readonly, unset and declare

```
readonly PI=3.14159        # constant, cannot be changed
PI=3                       # error: PI: readonly variable

x=10
unset x                    # delete variable
echo "${x:-not set}"       # prints: not set

declare -i num=5           # integer variable
num=num+10                 # arithmetic works without $(( ))
echo $num                  # 15

declare -r CONST="fixed"   # readonly
declare -x EXPORTED="yes"  # export
declare -l lower="HELLO"   # converts to lowercase on assignment -> hello
declare -u upper="hello"   # converts to uppercase -> HELLO
declare -a arr=(1 2 3)     # indexed array
declare -A map             # associative array
declare -p                 # print all variables with attributes
```

2.7 Parameter expansion (very powerful)

Bash can modify variable values directly using `${...}` without calling external programs. This is faster and used everywhere in professional scripts.

Syntax	Meaning	Example (file=/home/ravi/data.tar.gz)
`${var:-default}`	Use default if var is empty/unset	${name:-Guest}
`${var:=default}`	Assign default if empty/unset	${name:=Guest}
`${var:?message}`	Exit with error message if empty	${1:?Argument missing}
`${var:+alt}`	Use alt only if var is set	${debug:+--verbose}
`${#var}`	Length of string	${#file} -> 22
`${var:3}`	Substring from offset 3	${file:6} -> ravi/data.tar.gz
`${var:3:4}`	Substring offset 3, length 4	${file:6:4} -> ravi
`${var#pattern}`	Remove shortest match from start	${file#*/} -> home/ravi/data.tar.gz
`${var##pattern}`	Remove longest match from start	${file##*/} -> data.tar.gz
`${var%pattern}`	Remove shortest match from end	${file%.*} -> /home/ravi/data.tar
`${var%%pattern}`	Remove longest match from end	${file%%.*} -> /home/ravi/data
`${var/old/new}`	Replace first match	${file/ravi/amit}
`${var//old/new}`	Replace all matches	${file//a/A}
`${var^}`	First letter uppercase	x=hello; ${x^} -> Hello
`${var^^}`	All uppercase	${x^^} -> HELLO
`${var,}`	First letter lowercase	x=HELLO; ${x,} -> hELLO
`${var,,}`	All lowercase	${x,,} -> hello
`${!prefix*}`	Names of variables starting with prefix	${!BASH*}
`${!var}`	Indirect expansion (value of variable named by var)	a=b; b=5; ${!a} -> 5

expansion.sh

```
#!/bin/bash
path="/var/log/nginx/access.log.gz"
echo "Filename  : ${path##*/}"          # access.log.gz
echo "Directory : ${path%/*}"           # /var/log/nginx
echo "No ext    : ${path%.gz}"          # /var/log/nginx/access.log
echo "Extension : ${path##*.}"          # gz
name=${1:-World}                        # default argument
echo "Hello, $name!"
```

2.8 Reading values from the user

```
read -p "Enter your name: " name
echo "Welcome, $name"
```

More about `read` is in the next chapter.

2.9 Variable scope

By default variables are **global** in the script. Inside a function use `local` to make a variable private to the function. Variables set in a child process (another script, a subshell `( )`) never come back to the parent.

```
x=10
func() {
    local x=20          # only inside func
    y=30                # global (no local)
    echo "inside: x=$x"
}
func
echo "outside: x=$x y=$y"     # x=10 y=30

( z=5 )                       # runs in a subshell
echo "z=${z:-empty}"          # z=empty
```

2.10 Naming convention summary

- UPPER_CASE for constants and environment variables: `BACKUP_DIR`, `MAX_RETRY`.
- lower_case for ordinary and local variables: `file_name`, `count`.
- Use descriptive names: `backup_dir` is better than `bd`.
- Never use names of important variables by accident: `PATH`, `HOME`, `USER`, `IFS`.

Chapter 3: Input, Output and Redirection

3.1 echo

`echo` prints text. It is the most used command in scripts.

Option	Meaning	Example
`-n`	Do not print the newline at the end	`echo -n "Loading..."`
`-e`	Enable backslash escapes (\n \t \033)	`echo -e "Line1\nLine2"`
`-E`	Disable escapes (default)	`echo -E "a\nb"`

```
echo "Simple text"
echo -e "Name:\tRavi\nCity:\tBhilwara"
echo -n "No newline here"; echo " <- continues"
echo "Path is: $PATH"
echo "Multiple     spaces   preserved in quotes"
echo Multiple     spaces   collapsed without quotes
```

3.2 printf (formatted output)

`printf` works like C language printf. It is more reliable than `echo` for formatting and does not add a newline automatically.

Format	Meaning
`%s`	String
`%d or %i`	Integer
`%f / %.2f`	Floating number / with 2 decimals
`%x / %o`	Hexadecimal / octal
`%c`	Single character
`%5s / %-10s`	Right-aligned width 5 / left-aligned width 10
`%05d`	Zero padded number: 00042
`%%`	A literal percent sign
`\n \t`	New line / tab

```
printf "Hello %s, you are %d years old\n" "Ravi" 25
printf "%-10s %-8s %6s\n" "Name" "Dept" "Salary"
printf "%-10s %-8s %6.2f\n" "Amit" "IT" 45000.5
printf "%05d\n" 42                    # 00042
printf "Hex of 255 = %x\n" 255         # ff
printf '%s\n' "line1" "line2" "line3"  # prints each on new line
today=$(printf '%(%Y-%m-%d)T' -1)      # bash builtin date, no external command

Name       Dept     Salary
Amit       IT     45000.50
```

3.3 read (taking input)

Option	Meaning	Example
`-p "text"`	Show a prompt	`read -p "Name: " n`
`-s`	Silent (for passwords)	`read -s -p "Password: " pw`
`-t N`	Timeout after N seconds	`read -t 5 -p "Quick: " x`
`-n N`	Read only N characters	`read -n 1 -p "Press key" k`
`-r`	Raw: do not treat backslash as escape (always use)	`read -r line`
`-a arr`	Read words into an array	`read -a words`
`-d C`	Use C as delimiter instead of newline	`read -d "," part`
`-e`	Use readline editing (arrow keys work)	`read -e -p "File: " f`
`-i text`	Pre-filled default text (needs -e)	`read -e -i "yes" -p "OK? " a`

input.sh

```
#!/bin/bash
read -rp "Enter your name: " name
read -rsp "Enter password: " pass; echo
read -rp "Enter two numbers: " a b
echo "Hello $name, sum = $((a + b))"

# Yes / No question
read -rp "Continue? [y/N]: " ans
case "${ans,,}" in
    y|yes) echo "Continuing..." ;;
    *)     echo "Cancelled"; exit 1 ;;
esac

# read with timeout and default
read -t 5 -rp "Choose (5 sec): " choice || choice="default"
echo "Choice: $choice"
```

3.4 Redirection

Every program has three standard streams: **stdin (0)** input, **stdout (1)** normal output, **stderr (2)** error output. Redirection changes where they go.

Operator	Meaning	Example
`>`	Send stdout to file (overwrite)	`ls > list.txt`
`>>`	Append stdout to file	`date >> log.txt`
`<`	Take stdin from file	`wc -l < list.txt`
`2>`	Send stderr to file	`ls /nope 2> err.txt`
`2>>`	Append stderr to file	`cmd 2>> err.log`
`&>`	stdout and stderr to file	`cmd &> all.txt`
`&>>`	Append both to file	`cmd &>> all.txt`
`2>&1`	Send stderr to where stdout goes	`cmd > out.txt 2>&1`
`1>&2`	Send stdout to stderr (for error messages)	`echo "Error" 1>&2`
`> /dev/null`	Throw output away	`cmd > /dev/null`
`2> /dev/null`	Hide errors	`find / -name x 2> /dev/null`
`<<EOF`	Here document (multi-line input)	`cat <<EOF`
`<<<`	Here string (one string as input)	`bc <<< "2+3"`
`|`	Pipe: stdout of left to stdin of right	`ls | wc -l`
`|&`	Pipe both stdout and stderr	`cmd |& tee log`

> **WARNING:** Order matters: `cmd > file 2>&1` is correct. `cmd 2>&1 > file` does NOT send errors to the file.

```
ls /etc /nonexistent > out.txt 2> err.txt     # separate files
ls /etc /nonexistent > all.txt 2>&1           # same file
ls /etc /nonexistent &> all.txt               # shortcut (bash)
command_that_may_fail > /dev/null 2>&1        # silent
> emptyfile.txt                               # create/empty a file
```

3.5 Here documents and here strings

```
# Here document: multi-line text to a command or file
cat <<EOF > config.txt
Server : $HOSTNAME
User   : $USER
Date   : $(date +%F)
EOF

# Quote the marker to stop variable expansion
cat <<'EOF'
This prints $HOME literally
EOF

# <<- removes leading TABS so you can indent inside functions
if true; then
    cat <<-END
        indented text (tabs stripped)
    END
fi

# Here string
read -r first second <<< "hello world"
bc <<< "scale=2; 10/3"            # 3.33
grep -c "a" <<< "banana"
```

3.6 Pipes and tee

```
ps aux | grep nginx | grep -v grep          # find a process
cat /etc/passwd | cut -d: -f1 | sort        # sorted usernames
du -sh * | sort -rh | head -5               # 5 biggest items

# tee: show on screen AND save to file
ls -l | tee list.txt
ls -l | tee -a list.txt                     # append
echo "data" | sudo tee /etc/myfile > /dev/null   # write file as root
```

> **TIP:** `sudo echo "x" > /etc/file` fails because the redirect is done by YOUR shell, not by sudo. Use `echo "x" | sudo tee /etc/file` instead.

3.7 File descriptors and exec

```
# Redirect all output of the whole script to a log file
exec > >(tee -a /var/log/myscript.log) 2>&1

# Open file for reading on descriptor 3
exec 3< input.txt
read -r line <&3
echo "First line: $line"
exec 3<&-                  # close descriptor 3

# Write errors to stderr
echo "ERROR: disk full" >&2
```

3.8 Colors in the terminal

ANSI escape codes can colour the text. Use `tput` or raw codes.

```
RED='\033[0;31m'; GREEN='\033[0;32m'; YELLOW='\033[1;33m'
```

```
BLUE='\033[0;34m'; BOLD='\033[1m'; NC='\033[0m'     # NC = No Color

echo -e "${GREEN}SUCCESS${NC}: backup finished"
echo -e "${RED}ERROR${NC}: disk is full"
echo -e "${YELLOW}WARNING${NC}: low memory"
echo -e "${BOLD}${BLUE}Bold blue text${NC}"

# using tput
echo "$(tput setaf 2)green$(tput sgr0) $(tput bold)bold$(tput sgr0)"
tput clear        # clear screen
tput cup 5 10     # move cursor to row 5, col 10
```

Color	Code	Color	Code
Black	30	Magenta	35
Red	31	Cyan	36
Green	32	White	37
Yellow	33	Reset	0
Blue	34	Bold	1

3.9 Progress bar and spinner

progress.sh

```
#!/bin/bash
# Simple progress bar
for i in $(seq 1 20); do
    printf "\r[%-20s] %d%%" "$(printf '#%.0s' $(seq 1 $i))" $((i*5))
    sleep 0.1
done
echo

# Spinner while a command runs in background
sleep 5 &
pid=$!
chars='|/-\'
while kill -0 $pid 2>/dev/null; do
    for (( i=0; i<4; i++ )); do
        printf "\rWorking... %s" "${chars:$i:1}"
        sleep 0.15
    done
done
printf "\rDone!          \n"
```

Chapter 4: Operators and Expressions

4.1 Arithmetic

Bash does integer math only. Use `$(( ))` for calculations. For decimal (float) numbers use `bc` or `awk`.

Operator	Meaning	Example	Result
`+ -`	Add, subtract	`$((10 + 3))`	13
`* /`	Multiply, divide (integer)	`$((10 / 3))`	3
`%`	Remainder (modulus)	`$((10 % 3))`	1
`**`	Power	`$((2 ** 8))`	256
`++ --`	Increment / decrement	`((i++))`	i+1
`+= -= *=`	Compound assign	`((x += 5))`	x+5
`<< >>`	Bit shift	`$((1 << 4))`	16
`& | ^ ~`	Bitwise AND OR XOR NOT	`$((6 & 3))`	2

math.sh

```
a=10; b=3
echo "Sum        : $((a + b))"
echo "Difference : $((a - b))"
echo "Product    : $((a * b))"
echo "Quotient   : $((a / b))"
echo "Remainder  : $((a % b))"
echo "Power      : $((a ** 2))"

((a++));  echo $a            # 11
((a += 5)); echo $a          # 16
let "c = a * 2"; echo $c     # 32  (let command)
d=$(expr 5 + 3); echo $d     # 8   (old style expr)

# Floating point with bc and awk
echo "scale=2; 10/3" | bc               # 3.33
echo "scale=4; sqrt(2)" | bc -l         # 1.4142
awk 'BEGIN { printf "%.2f\n", 10/3 }'   # 3.33
result=$(echo "scale=2; 22/7" | bc); echo $result
```

4.2 Numeric comparison

Inside `[ ]` or `[[ ]]` use these word operators for numbers. Inside `(( ))` you can use normal symbols.

Operator	(( )) form	Meaning
`-eq`	`==`	Equal
`-ne`	`!=`	Not equal
`-gt`	`>`	Greater than
`-ge`	`>=`	Greater than or equal
`-lt`	`<`	Less than
`-le`	`<=`	Less than or equal

```
a=5; b=8
[ $a -lt $b ] && echo "a is smaller"
[[ $a -eq 5 ]] && echo "a is five"
(( a < b )) && echo "also smaller"
if (( a != b )); then echo "different"; fi
```

4.3 String comparison

Operator	Meaning	Example
`= or ==`	Strings equal	`[[ $a == "yes" ]]`
`!=`	Strings not equal	`[[ $a != "no" ]]`
`-z str`	String is empty (zero length)	`[[ -z $name ]]`
`-n str`	String is not empty	`[[ -n $name ]]`
`< >`	Alphabetical order (inside [[ ]])	`[[ apple < banana ]]`
`== pattern`	Glob pattern match (unquoted, in [[ ]])	`[[ $f == *.txt ]]`
`=~`	Regex match (in [[ ]])	`[[ $x =~ ^[0-9]+$ ]]`

```
name="ravi"
[[ $name == "ravi" ]] && echo "Hi Ravi"
[[ -z "$unset_var" ]] && echo "variable is empty"
[[ $name == r* ]] && echo "starts with r"
[[ "123" =~ ^[0-9]+$ ]] && echo "only digits"
```

4.4 File test operators

Test	True if...
`-e file`	file or directory exists
`-f file`	it is a regular file
`-d file`	it is a directory
`-L file`	it is a symbolic link
`-r -w -x`	readable / writable / executable by you
`-s file`	file exists and size is greater than zero
`-b -c -p -S`	block device / char device / named pipe / socket
`-O -G`	owned by your user / your group
`-u -g -k`	setuid / setgid / sticky bit is set
`f1 -nt f2`	f1 is newer than f2
`f1 -ot f2`	f1 is older than f2
`f1 -ef f2`	same file (same inode)
`-t fd`	file descriptor is open on a terminal (e.g. -t 1)

```
file="/etc/passwd"
if [[ -f $file ]]; then echo "$file is a file"; fi
if [[ -d /tmp ]];  then echo "/tmp is a directory"; fi
if [[ -r $file && ! -w $file ]]; then echo "read-only for me"; fi
[[ -s backup.tar.gz ]] || echo "backup is empty or missing!"
```

4.5 Logical operators

Operator	Meaning	Example
`&&`	AND: run right command only if left succeeded	`mkdir d && cd d`
`||`	OR: run right command only if left failed	`cd d || exit 1`
`!`	NOT	`[[ ! -f x ]]`
`-a -o`	AND / OR inside old [ ] (avoid)	`[ $a -gt 1 -a $b -lt 5 ]`
`&& ||`	AND / OR inside [[ ]]	`[[ $a -gt 1 && $b -lt 5 ]]`
`;`	Run commands one after another	`cd /tmp; ls`
`&`	Run command in background	`sleep 60 &`

```
# Short-circuit style
mkdir -p /tmp/work && echo "created" || echo "failed"
[[ -f config.cfg ]] || { echo "config missing"; exit 1; }
command -v git >/dev/null 2>&1 || echo "git is not installed"

# Combined conditions
age=30; city="Bhilwara"
if [[ $age -ge 18 && $city == "Bhilwara" ]]; then echo "Eligible"; fi
if [[ $age -lt 18 || $age -gt 60 ]]; then echo "Out of range"; fi
```

4.6 [ ] versus [[ ]] versus (( ))

Form	Type	Notes
`[ ... ]`	POSIX test command	Works in sh and bash. Must quote variables. Use -a -o && inside is awkward.
`[[ ... ]]`	Bash keyword	Safer: no word splitting, supports &&, ||, =~ and patterns. PREFER in bash scripts.
`(( ... ))`	Arithmetic	C-like math and comparisons with ==, <, >, etc.
`test expr`	Same as [ ]	Old style: test -f file

4.7 Operator precedence quick note

Use parentheses to be clear. In `(( ))` the order is the same as in C: `!`, `* / %`, `+ -`, comparison, `&&`, `||`. In `[[ ]]` group with `( )`: `[[ ( $a -gt 1 || $b -gt 1 ) && -f file ]]`.

4.8 Example: simple calculator

calc.sh

```
#!/bin/bash
read -rp "Enter first number : " n1
read -rp "Enter operator (+ - * /): " op
read -rp "Enter second number: " n2

case $op in
    +) res=$(echo "scale=2; $n1 + $n2" | bc) ;;
    -) res=$(echo "scale=2; $n1 - $n2" | bc) ;;
    \*|x) res=$(echo "scale=2; $n1 * $n2" | bc) ;;
    /) if [[ $n2 == 0 ]]; then echo "Cannot divide by zero"; exit 1; fi
       res=$(echo "scale=2; $n1 / $n2" | bc) ;;
    *) echo "Invalid operator"; exit 1 ;;
esac
echo "Result: $n1 $op $n2 = $res"
```

Chapter 5: Decision Making (if, case)

5.1 if / elif / else

The `if` statement runs a command and checks its exit status. If it is 0 (success) the `then` block runs. Always finish with `fi`. Note the spaces inside `[[ ]]`.

syntax

```
if condition; then
    commands
elif other_condition; then
    commands
else
    commands
fi
```

grade.sh

```
#!/bin/bash
read -rp "Enter your marks (0-100): " marks

if   (( marks >= 90 )); then grade="A+"
elif (( marks >= 75 )); then grade="A"
elif (( marks >= 60 )); then grade="B"
elif (( marks >= 40 )); then grade="C"
else                         grade="Fail"
fi
echo "Your grade: $grade"
```

5.2 if with commands (no brackets)

`if` can test any command, not only `[ ]`. Very useful with `grep`, `ping`, `diff`, `systemctl`, etc.

```
if grep -q "root" /etc/passwd; then
    echo "root user exists"
fi

if ping -c 1 -W 2 google.com > /dev/null 2>&1; then
    echo "Internet is working"
else
    echo "No internet"
fi

if ! command -v docker > /dev/null; then
    echo "Docker not installed"
fi

if systemctl is-active --quiet nginx; then
    echo "nginx running"
fi

if cmp -s file1 file2; then echo "files are same"; fi
```

5.3 Common one-line checks

```
# Must be root
if [[ $EUID -ne 0 ]]; then
    echo "Please run as root: sudo $0" >&2
    exit 1
fi

# Argument required
if [[ $# -lt 1 ]]; then echo "Usage: $0 filename"; exit 1; fi

# File exists check
[[ -f "$1" ]] || { echo "File not found: $1"; exit 1; }

# Even or odd
n=7; (( n % 2 == 0 )) && echo even || echo odd

# Is a number? (integer check)
[[ $x =~ ^-?[0-9]+$ ]] && echo "number" || echo "not a number"
```

5.4 case statement

`case` compares one value with many patterns. It is cleaner than many `elif` lines. Each pattern ends with `)` and each block ends with `;;`. Use `|` for OR and `*)` for default.

syntax

```
case "$variable" in
    pattern1)  commands ;;
    pattern2|pattern3)  commands ;;
    *)  default commands ;;
esac
```

case.sh

```
#!/bin/bash
read -rp "Enter a day name: " day
case "${day,,}" in
    monday|tuesday|wednesday|thursday|friday)
        echo "Weekday - go to work" ;;
    saturday|sunday)
        echo "Weekend - relax" ;;
    *)
        echo "Not a valid day" ;;
esac

# Pattern matching with case
case "$1" in
    *.txt)        echo "Text file" ;;
    *.jpg|*.png)  echo "Image file" ;;
    [0-9]*)       echo "Starts with digit" ;;
    ?)            echo "Single character" ;;
    *)            echo "Unknown" ;;
esac
```

Terminator	Meaning
`;;`	End of this case, leave the case statement
`;&`	Continue and run the NEXT block without testing its pattern
`;;&`	Continue testing the next patterns

5.5 Service control menu with case

service.sh

```
#!/bin/bash
# usage: ./service.sh {start|stop|restart|status}
SERVICE="nginx"
case "$1" in
    start)   sudo systemctl start   "$SERVICE" && echo "$SERVICE started"  ;;
    stop)    sudo systemctl stop    "$SERVICE" && echo "$SERVICE stopped"  ;;
    restart) sudo systemctl restart "$SERVICE" && echo "$SERVICE restarted";;
    status)  systemctl status "$SERVICE" --no-pager ;;
    *)       echo "Usage: $0 {start|stop|restart|status}"; exit 1 ;;
esac
```

5.6 Practical examples

Leap year check

leap.sh

```
#!/bin/bash
read -rp "Enter year: " y
if (( (y % 4 == 0 && y % 100 != 0) || y % 400 == 0 )); then
    echo "$y is a leap year"
else
    echo "$y is not a leap year"
fi
```

Largest of three numbers

largest.sh

```
#!/bin/bash
read -rp "Enter three numbers: " a b c
if (( a >= b && a >= c )); then max=$a
elif (( b >= a && b >= c )); then max=$b
else max=$c
fi
echo "Largest = $max"
```

Check disk usage and warn

diskcheck.sh

```
#!/bin/bash
usage=$(df / | awk 'NR==2 {print $5}' | tr -d '%')
if   (( usage >= 90 )); then echo "CRITICAL: root disk ${usage}% full"
elif (( usage >= 75 )); then echo "WARNING : root disk ${usage}% full"
else                         echo "OK      : root disk ${usage}% used"
fi
```

Password strength checker

pwcheck.sh

```
#!/bin/bash
read -rsp "Enter password: " pw; echo
score=0
(( ${#pw} >= 8 ))      && ((score++))
[[ $pw =~ [A-Z] ]]     && ((score++))
[[ $pw =~ [a-z] ]]     && ((score++))
[[ $pw =~ [0-9] ]]     && ((score++))
[[ $pw =~ [^a-zA-Z0-9] ]] && ((score++))
case $score in
    5) echo "Very strong" ;;
    4) echo "Strong" ;;
    3) echo "Medium" ;;
    *) echo "Weak" ;;
esac
```

5.7 Common mistakes

Mistake	Problem	Fix
`if [$a -eq 1]`	No spaces inside brackets	if [$a -eq 1]
`if [ $a = $b ]`	Breaks when variable is empty	if ["$a" = "$b"] or [[$a == $b]]
`if [ $a == 1 ]`	== inside [ ] is not POSIX	Use = or [[ ]]
`if [ $a > 5 ]`	Creates a file named 5!	Use -gt or ((a > 5))
`if (( $x = 5 ))`	Assignment not comparison	Use == : ((x == 5))
`Forgetting then/fi`	Syntax error	Every if needs then and fi

Chapter 6: Loops

Loops repeat commands. Bash has `for`, `while`, `until` and `select` loops. Use `break` to leave a loop and `continue` to skip to the next round.

6.1 for loop (list)

```
for fruit in apple banana cherry; do
    echo "I like $fruit"
done

for i in 1 2 3 4 5; do echo "Number $i"; done

for i in {1..10}; do echo -n "$i "; done; echo         # brace range
for i in {0..100..10}; do echo -n "$i "; done; echo   # step of 10
for c in {a..e}; do echo -n "$c "; done; echo         # letters
for i in $(seq 1 2 9); do echo -n "$i "; done; echo   # 1 3 5 7 9
```

6.2 for loop (C style)

```
for (( i=1; i<=5; i++ )); do
    echo "Count: $i"
done

for (( i=10; i>0; i-=2 )); do echo -n "$i "; done; echo

# two variables
for (( i=0, j=10; i<j; i++, j-- )); do echo "$i $j"; done
```

6.3 for loop over files and command output

```
# All .txt files in current directory
for f in *.txt; do
    [[ -e $f ]] || continue          # skip if no match
    echo "Processing $f ($(wc -l < "$f") lines)"
done

# Files in a directory recursively
for f in /var/log/*.log; do echo "$f"; done

# Arguments of the script
for arg in "$@"; do echo "Arg: $arg"; done

# Output of a command (lines without spaces)
for user in $(cut -d: -f1 /etc/passwd | head -5); do echo "User: $user"; done

# Directories only
for d in */; do echo "Dir: ${d%/}"; done
```

> **WARNING:** Never use `for f in $(ls)`: it breaks on file names with spaces. Use `for f in *` or `find ... -print0 | while read -r -d ''`.

6.4 while loop

```
count=1
while [ $count -le 5 ]; do
    echo "Count = $count"
    ((count++))
done

# Infinite loop with break
while true; do
    read -rp "Type quit to stop: " x
    [[ $x == quit ]] && break
    echo "You typed: $x"
done

# Wait until a file appears
while [[ ! -f /tmp/ready.flag ]]; do
    echo "waiting..."; sleep 2
done
```

6.5 Reading a file line by line

The correct and safe way is `while IFS= read -r line`. `IFS=` keeps leading spaces, `-r` keeps backslashes.

readfile.sh

```
#!/bin/bash
file="servers.txt"
n=0
while IFS= read -r line; do
    [[ -z $line || $line == \#* ]] && continue    # skip blank and comment lines
    ((n++))
    echo "$n: $line"
done < "$file"

# Read CSV: name,age,city
while IFS=',' read -r name age city; do
    echo "Name=$name Age=$age City=$city"
done < people.csv

# Read output of a command (no subshell problem)
while read -r pid cmd; do echo "PID $pid -> $cmd"; done < <(ps -eo pid,comm | head -5)
```

> **TIP:** `cmd | while read ...` runs the loop in a subshell, so variables changed inside are lost after the loop. Use `while read ...; done < <(cmd)` (process substitution) instead.

6.6 until loop

`until` runs while the condition is **false** (opposite of while).

```
n=1
until [ $n -gt 5 ]; do
    echo "n = $n"; ((n++))
done

# Retry until command succeeds
until ping -c1 -W1 8.8.8.8 &>/dev/null; do
    echo "Network down, retrying in 5 sec..."; sleep 5
done
echo "Network is up"
```

6.7 select loop (menu)

select_menu.sh

```
#!/bin/bash
PS3="Choose an option (number): "
select choice in "Show date" "Show disk" "Show memory" "Quit"; do
    case $choice in
        "Show date")   date ;;
        "Show disk")   df -h / ;;
        "Show memory") free -h ;;
        "Quit")        echo "Bye"; break ;;
        *)             echo "Invalid choice: $REPLY" ;;
    esac
done
```

6.8 break and continue

```
for i in {1..10}; do
    (( i == 3 )) && continue       # skip 3
    (( i == 7 )) && break          # stop at 7
    echo -n "$i "
done; echo                          # 1 2 4 5 6

# break N breaks N levels of nested loops
for i in 1 2 3; do
    for j in 1 2 3; do
        (( j == 2 )) && continue 2  # continue outer loop
        echo "$i-$j"
    done
done
```

6.9 Practical loop programs

Multiplication table

table.sh

```
#!/bin/bash
read -rp "Table of: " n
for i in {1..10}; do printf "%d x %2d = %3d\n" "$n" "$i" $((n*i)); done
```

Factorial

factorial.sh

```
#!/bin/bash
read -rp "Enter number: " n
f=1
for (( i=2; i<=n; i++ )); do f=$((f*i)); done
echo "$n! = $f"
```

Fibonacci series

fibonacci.sh

```
#!/bin/bash
read -rp "How many terms? " n
a=0; b=1
for (( i=0; i<n; i++ )); do
    echo -n "$a "
    t=$((a+b)); a=$b; b=$t
done; echo
```

Prime number check

prime.sh

```
#!/bin/bash
read -rp "Enter number: " n
(( n < 2 )) && { echo "Not prime"; exit; }
for (( i=2; i*i<=n; i++ )); do
    if (( n % i == 0 )); then echo "$n is NOT prime"; exit; fi
done
echo "$n is prime"
```

Sum of digits and reverse a number

digits.sh

```
#!/bin/bash
read -rp "Enter number: " n
sum=0; rev=0; t=$n
while (( t > 0 )); do
    d=$((t % 10)); sum=$((sum + d)); rev=$((rev*10 + d)); t=$((t / 10))
done
echo "Sum of digits: $sum   Reverse: $rev"
```

Star pattern

pyramid.sh

```
#!/bin/bash
read -rp "Rows: " r
for (( i=1; i<=r; i++ )); do
    printf '%*s' $((r-i)) ''            # leading spaces
    for (( j=1; j<=2*i-1; j++ )); do printf '*'; done
    echo
done
```

```
Rows: 4
   *
  ***
 *****
*******
```

Countdown timer

countdown.sh

```
#!/bin/bash
for (( s=${1:-10}; s>=0; s-- )); do
    printf "\rTime left: %02d sec " "$s"; sleep 1
done; echo -e "\nTime up!"
```

Bulk rename files

rename.sh

```
#!/bin/bash
# Rename all *.jpeg to *.jpg and add numbering
n=1
for f in *.jpeg; do
    [[ -e $f ]] || continue
    new=$(printf "photo_%03d.jpg" "$n")
    mv -v -- "$f" "$new"
    ((n++))
done
```

Ping a list of servers

pingall.sh

```
#!/bin/bash
for ip in 192.168.1.{1..10}; do
    if ping -c1 -W1 "$ip" &>/dev/null; then echo "$ip is UP"
    else echo "$ip is down"; fi
done
```

6.10 Running loop bodies in parallel

```
# Start jobs in background and wait for all
for f in *.log; do
    gzip "$f" &
done
wait
echo "All files compressed"

# Limit to 4 parallel jobs with xargs
ls *.log | xargs -P 4 -I{} gzip {}
```

Chapter 7: Functions

A function is a named block of code that you can call many times. Functions make scripts short, clean and easy to fix. Define a function **before** you call it.

7.1 Defining and calling

```
greet() {
    echo "Hello from function"
}
greet                       # call it

# Second style (also valid)
function bye {
    echo "Goodbye"
}
bye
```

7.2 Arguments

Inside a function, `$1 $2 ...`, `$#`, `$@` refer to the **function arguments**, not the script arguments.

```
greet() {
    local name="$1"
    local city="${2:-unknown}"
    echo "Hello $name from $city (got $# args)"
}
greet "Ravi" "Bhilwara"
greet "Amit"
```

7.3 Returning values

`return` only gives a number 0-255 (exit status). To return text or big numbers, `echo` the value and capture it with `$( )`, or set a global variable.

```
is_even() { (( $1 % 2 == 0 )); }          # status is the result
if is_even 10; then echo "10 is even"; fi

add() { echo $(( $1 + $2 )); }            # output is the result
sum=$(add 15 27)
echo "Sum = $sum"

get_ip() { hostname -I | awk '{print $1}'; }
echo "My IP: $(get_ip)"

# Return via nameref (bash 4.3+)
compute() { local -n out=$1; out=$(( $2 * $3 )); }
compute result 6 7
echo "$result"            # 42
```

7.4 Local variables

```
counter=0
increment() {
    local step=${1:-1}      # local: invisible outside
    counter=$((counter + step))
}
increment; increment 5
echo "$counter"             # 6
```

> **WARNING:** Always use `local` for function variables. Without it, a function can silently overwrite a variable of the main script.

7.5 Recursion

```
factorial() {
    local n=$1
    if (( n <= 1 )); then echo 1
    else
        local prev=$(factorial $((n - 1)))
        echo $(( n * prev ))
    fi
}
echo "5! = $(factorial 5)"        # 120

# Directory tree sizes
walk() {
    for item in "$1"/*; do
        [[ -d $item ]] && { echo "DIR  $item"; walk "$item"; }
    done
}
```

7.6 Useful function library

These small functions are used in almost every real script. Save them in `lib.sh` and use `source lib.sh`.

lib.sh

```
#!/bin/bash
# lib.sh - common helper functions
RED=$'\033[31m'; GRN=$'\033[32m'; YEL=$'\033[33m'; NC=$'\033[0m'

log()   { echo "$(date '+%F %T') $*"; }
info()  { echo "${GRN}[INFO]${NC}  $*"; }
warn()  { echo "${YEL}[WARN]${NC}  $*" >&2; }
error() { echo "${RED}[ERROR]${NC} $*" >&2; }
die()   { error "$*"; exit 1; }

need_root() { [[ $EUID -eq 0 ]] || die "Run as root (use sudo)"; }
need_cmd()  { command -v "$1" >/dev/null 2>&1 || die "Required command not found: $1"; }

confirm() {                      # confirm "Delete all?" && rm ...
    local ans
    read -rp "$1 [y/N]: " ans
    [[ ${ans,,} == y || ${ans,,} == yes ]]
}

is_number() { [[ $1 =~ ^-?[0-9]+$ ]]; }

file_size() { stat -c %s "$1"; }          # bytes

human() { numfmt --to=iec --suffix=B "$1"; }   # 1048576 -> 1.0MiB

retry() {                         # retry 3 curl -s url
    local n=$1; shift
    local i
    for (( i=1; i<=n; i++ )); do
        "$@" && return 0
        warn "Attempt $i/$n failed: $*"; sleep $((i*2))
    done
    return 1
}

timestamp() { date +%Y%m%d_%H%M%S; }
```

use_lib.sh

```
#!/bin/bash
source ./lib.sh
need_cmd curl
confirm "Start download?" || die "Cancelled"
retry 3 curl -fsS -o page.html https://example.com && info "Downloaded"
```

7.7 Function tips

- Function names should be verbs: `backup_files`, `check_disk`, `send_alert`.
- List functions: `declare -F`. Show code of one: `declare -f name`.
- Remove a function: `unset -f name`. Export to child bash: `export -f name`.
- A function can read piped input: `upper() { tr a-z A-Z; }; echo hi | upper`.
- Put `main "$@"` at the bottom so all functions are defined before use.

Chapter 8: Arrays

8.1 Indexed arrays

```
fruits=("apple" "banana" "cherry")        # create
fruits[3]="mango"                         # add by index
fruits+=("grape" "kiwi")                  # append

echo "${fruits[0]}"                       # apple
echo "${fruits[@]}"                       # all elements
echo "${#fruits[@]}"                      # number of elements: 6
echo "${!fruits[@]}"                      # all indexes: 0 1 2 3 4 5
echo "${fruits[@]:1:3}"                   # slice: banana cherry mango
echo "${fruits[-1]}"                      # last element: kiwi
unset 'fruits[1]'                         # delete one element
echo "${#fruits[2]}"                      # length of element 2 string
```

8.2 Looping through arrays

```
servers=("web1" "web2" "db1")

for s in "${servers[@]}"; do echo "Server: $s"; done

for i in "${!servers[@]}"; do echo "Index $i = ${servers[$i]}"; done

i=0
while (( i < ${#servers[@]} )); do echo "${servers[i]}"; ((i++)); done
```

WARNING: Always write `"${array[@]}"` with quotes. Without quotes, elements containing spaces are split.

8.3 Array operations

```
nums=(5 3 9 1 7)

# sort
sorted=($(printf '%s\n' "${nums[@]}" | sort -n))
echo "${sorted[@]}"                       # 1 3 5 7 9

# sum and max
sum=0; max=${nums[0]}
for n in "${nums[@]}"; do (( sum += n )); (( n > max )) && max=$n; done
echo "sum=$sum max=$max"

# check if element exists
contains() { local e; for e in "${@:2}"; do [[ $e == "$1" ]] && return 0; done; return 1; }
contains 9 "${nums[@]}" && echo "9 found"

# copy array
copy=("${nums[@]}")

# join with comma
IFS=, ; echo "${nums[*]}" ; IFS=$' \t\n'

# read file into array
mapfile -t lines < /etc/hosts             # one line per element
echo "Lines in file: ${#lines[@]}"
readarray -t users < <(cut -d: -f1 /etc/passwd)

# split string into array
IFS=':' read -ra parts <<< "$PATH"
echo "PATH has ${#parts[@]} directories"
```

8.4 Associative arrays (key-value)

Bash 4+ supports dictionaries. You **must** declare them with `declare -A`.

```
declare -A age
```

```
age[Ravi]=25
age[Amit]=30
age["Neha Sharma"]=28

echo "${age[Ravi]}"                 # 25
echo "${!age[@]}"                   # all keys
echo "${age[@]}"                    # all values
echo "${#age[@]}"                   # count

for name in "${!age[@]}"; do echo "$name is ${age[$name]} years old"; done

[[ -v age[Amit] ]] && echo "Amit exists"       # key check
unset 'age[Amit]'                               # delete key

# create with values
declare -A colors=([red]="#f00" [green]="#0f0" [blue]="#00f")
```

Example: word frequency counter

wordfreq.sh

```
#!/bin/bash
declare -A freq
for w in $(tr -s '[:space:][:punct:]' '\n' < "$1" | tr 'A-Z' 'a-z'); do
    (( freq[$w]++ ))
done
for w in "${!freq[@]}"; do echo "${freq[$w]} $w"; done | sort -rn | head -10
```

Example: count file types

exts.sh

```
#!/bin/bash
declare -A ext
for f in *.*; do (( ext[${f##*.}]++ )); done
for e in "${!ext[@]}"; do printf "%-6s %d\n" "$e" "${ext[$e]}"; done
```

Chapter 9: String Manipulation

9.1 Basic string operations

```
s="Hello, Ubuntu World"
echo "${#s}"                        # length: 19
echo "${s:7:6}"                     # substring: Ubuntu
echo "${s: -5}"                     # last 5 chars: World  (note the space)
echo "${s/Ubuntu/Linux}"            # replace first: Hello, Linux World
echo "${s//o/0}"                    # replace all: Hell0, Ubuntu W0rld
echo "${s^^}"                       # UPPERCASE
echo "${s,,}"                       # lowercase
echo "${s#Hello, }"                 # remove prefix: Ubuntu World
echo "${s% World}"                  # remove suffix: Hello, Ubuntu
```

9.2 Checking strings

```
s="backup_2026.tar.gz"
[[ $s == backup* ]]  && echo "starts with backup"
[[ $s == *.gz ]]     && echo "ends with .gz"
[[ $s == *2026* ]]   && echo "contains 2026"
[[ -z $s ]] && echo "empty"
[[ $s =~ ^backup_([0-9]{4})\.tar\.gz$ ]] && echo "Year = ${BASH_REMATCH[1]}"
[[ "abc" < "abd" ]] && echo "abc comes first"
```

9.3 Trimming, padding, repeating

```
s="   hello world   "
trimmed=$(echo "$s" | xargs)                      # simple trim
trim() { local v="$*"; v="${v#"${v%%[![:space:]]*}"}"; v="${v%"${v##*[![:space:]]}"}"; echo "$v"; }
echo "[$(trim "$s")]"                             # [hello world]

printf "%-10s|\n" "left"                          # pad right
printf "%10s|\n" "right"                          # pad left
printf '=%.0s' {1..40}; echo                      # repeat "=" 40 times
line=$(printf '%*s' 30 '' | tr ' ' '-'); echo "$line"
```

9.4 Splitting and joining

```
csv="apple,banana,cherry"
IFS=',' read -ra items <<< "$csv"
echo "${items[1]}"                  # banana
echo "${#items[@]}"                 # 3

# join array with a separator
join_by() { local IFS="$1"; shift; echo "$*"; }
join_by ' | ' "${items[@]}"         # apple | banana | cherry

# cut, awk, tr
echo "a:b:c" | cut -d: -f2          # b
echo "John Smith" | awk '{print $2}'
echo "hello" | tr 'a-z' 'A-Z'       # HELLO
echo "hello" | rev                  # olleh
```

9.5 Useful string examples

Palindrome check

palindrome.sh

```
#!/bin/bash
read -rp "Enter a word: " w
w=${w,,}
[[ $w == "$(echo "$w" | rev)" ]] && echo "Palindrome" || echo "Not palindrome"
```

Count vowels

vowels.sh

```
#!/bin/bash
read -rp "Enter text: " t
v=$(echo "$t" | grep -io '[aeiou]' | wc -l)
echo "Vowels: $v"
```

Random password generator

genpass.sh

```
#!/bin/bash
len=${1:-16}
tr -dc 'A-Za-z0-9!@#$%^&*' < /dev/urandom | head -c "$len"; echo
# alternative: openssl rand -base64 12
```

Convert file names to lowercase

tolower.sh

```
#!/bin/bash
for f in *; do
    lower="${f,,}"
    [[ $f != "$lower" ]] && mv -v -- "$f" "$lower"
done
```

Extract parts of a URL

```
url="https://www.example.com:8080/path/page.html?id=5"
proto="${url%%://*}"                    # https
rest="${url#*://}"                      # www.example.com:8080/path/...
hostport="${rest%%/*}"                  # www.example.com:8080
host="${hostport%%:*}"                  # www.example.com
echo "$proto  $host"
```